

Lab Report No 8
Understanding of BFS and DFS algorithms
Artificial Intelligence



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

13th May, 2025

Department of Computer Engineering,
HITEC University, Taxila

Lab Task No 1:

Solution:

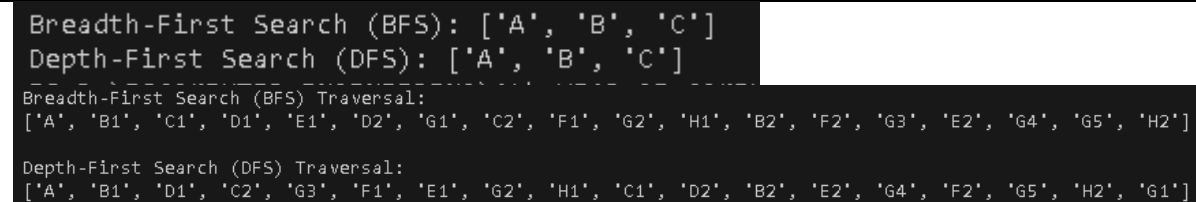
Brief description (3-5 lines)
Breadth-First Search: BFS stands for Breadth-First Search. It is a graph traversal algorithm used in computer science for traversing or searching tree or graph data structures. The purpose of BFS is to systematically visit and explore all the nodes of a graph or tree by exploring all the neighbor nodes at the present depth level before moving on to the nodes at the next depth level. Depth-First Search: DFS stands for Depth-First Search. It is another graph traversal algorithm used in computer science to traverse or search tree or graph data structures. Unlike BFS, which systematically explores all neighbor nodes at the present depth level before moving on to the nodes at the next depth level, DFS explores as far as possible along each branch before backtracking. Write a Python program for the breath first search and depth first search for given graph.
The code
<pre>from collections import deque # Define the tree structure based on the image using unique node labels graph = { 'A': ['B1', 'C1'], 'B1': ['D1', 'E1'], 'C1': ['D2', 'G1'], 'D1': ['C2', 'F1'], 'E1': ['G2', 'H1'], 'D2': ['B2', 'F2'], 'C2': ['G3'], 'F1': [], 'B2': ['E2', 'G4'], 'F2': ['G5', 'H2'], 'G1': [], 'G2': [], 'H1': [], 'E2': [], 'G3': [], 'G4': [], 'G5': [], 'H2': [] } # Breadth-First Search (BFS) def bfs(graph, start_node):</pre>

```

visited = []
queue = deque([start_node])
while queue:
    node = queue.popleft()
    if node not in visited:
        visited.append(node)
        queue.extend([neighbor for neighbor in graph.get(node, []) if neighbor
not in visited and neighbor not in queue])
    return visited
# Depth-First Search (DFS)
def dfs(graph, start_node, visited=None):
    if visited is None:
        visited = []
    if start_node not in visited:
        visited.append(start_node)
        for neighbor in graph.get(start_node, []):
            dfs(graph, neighbor, visited)
    return visited
# Main execution
if __name__ == "__main__":
    start = 'A'
    print("Breadth-First Search (BFS) Traversal:")
    print(bfs(graph, start))
    print("\nDepth-First Search (DFS) Traversal:")
    print(dfs(graph, start))

```

The results (Screenshot)



```

Breadth-First Search (BFS): ['A', 'B', 'C']
Depth-First Search (DFS): ['A', 'B', 'C']
-----
Breadth-First Search (BFS) Traversal:
['A', 'B1', 'C1', 'D1', 'E1', 'D2', 'G1', 'C2', 'F1', 'G2', 'H1', 'B2', 'F2', 'G3', 'E2', 'G4', 'G5', 'H2']

Depth-First Search (DFS) Traversal:
['A', 'B1', 'D1', 'C2', 'G3', 'F1', 'E1', 'G2', 'H1', 'C1', 'D2', 'B2', 'E2', 'G4', 'F2', 'G5', 'H2', 'G1']

```

Conclusion

In conclusion, we implemented the algorithms of breadth first search and depth first search. By working and operations by using python and we check the behavior of breadth first search and depth first search.